

JalanKL MVP — Development Plan

2026-09-18 · @Someone

1. MVP on a page

JalanKL is a phone web app that guides a first-time tourist from KLIA into KL, around six City Centre landmarks, and rewards real visits with stamps and postcards. We build it from scratch in about 4 weeks with 3 people.

The bet: tourists will follow one clear recommended route (not a list of options), tap "I'm Here" at each step, and come back for the stamp and postcard.

In the MVP	Not in the MVP
Arrival journey: KLIA to KL Sentral	3D mini city (we use a tilted 2D map instead)
City Centre: 6 landmarks + KL Sentral as check-in spots	Automatic route finder (we hand-write route cards)
Hand-written route cards with "I'm Here" steps	Login and accounts
One recommended option per route, plus "Open in Google Maps"	Live train times
GPS-confirmed check-in, gold and outline stamps	Languages beyond English and Malay
Landmark icon turns from grey to colour on the map	Printed or posted postcards
Review form, then postcard sent by email (3 landmarks first)	Partner rewards, audio guides, itinerary builder
Transit basics pages (Touch 'n Go, line colours, signage photos)	Other neighbourhoods
Nationality picker and neutral silhouette avatar	Custom mascot
English and Malay	Gender and other profile questions
Installable from a link or QR code	App store release

Done means: a first-time visitor with a mid-range Android phone or an iPhone can open the link at KLIA, follow the steps to KL Sentral, travel to Petronas Twin Towers, check in there, get a gold stamp, see the icon coloured on the map, submit a review, and receive the postcard by email. The stamp is still there after closing and reopening the app.

2. Decisions and assumptions

These merge the original PDF plan with the team's feature notes. Rows marked "assumed" were not confirmed by the team, so change them here if they are wrong.

Topic	Decision	Status
Codebase	Build from scratch. No existing Jalan KL code.	Confirmed
Area	Arrival (KLIA to KL Sentral) plus City Centre	Confirmed
Rewards	Both stamps and emailed postcards	Confirmed
Map	Tilted 2D map (2.5D look) with landmark icons. No 3D city.	Agreed
Routes	About 12 hand-written route cards stored as data. No route finder.	Agreed
Check-in	User taps "I'm Here", then the app checks GPS is inside the landmark radius	Agreed
Languages	English and Malay. Built so a new language is just a new file.	Assumed
Postcards	3 landmarks first: Petronas Twin Towers, Sultan Abdul Samad, Petaling Street	Assumed
Deadline	About 4 weeks from kickoff	Assumed
Team skills	All three can write basic HTML and JavaScript. Danial takes the lightest coding load.	Assumed
Muzium Negara	Optional 8th check-in spot, only if week 2 is on track	Assumed
KrackedDev	Role comes later. Not needed to build the MVP.	Confirmed
AI assistant	Code is written with an AI coding assistant, so this doc is written to be pasted into it	Confirmed

Check-in spots (7): KL Sentral, Petronas Twin Towers, KLCC Park, KL Tower, Merdeka 118, Sultan Abdul Samad Building, Petaling Street.

Why arrival steps do not need GPS: inside KLIA and on trains, GPS is weak or missing. Arrival steps advance on the "I'm Here" tap alone. Only the 7 check-in spots ask for GPS proof.

3. User flows

The whole app is one loop: pick a place, follow the steps, check in, get rewarded.

flowchart TD

```

A[Open link or QR] --> B[Pick language<br/>and nationality]
B --> C{Where are you?}
C -->|Just landed| D[Arrival journey<br/>KLIA to KL Sentral]
C -->|Already in KL| E[City Centre list]
D --> E
E --> F[Landmark detail]
F --> G[Journey steps<br/>tap I'm Here]
G --> H[Check-in<br/>GPS confirms]
H --> I[Gold stamp +<br/>icon coloured]
I --> J[Write a review]
J --> K[Postcard by email]
I --> E
  
```

After a stamp, the visitor can review for a postcard or go straight back to the list for the next landmark.

Flow	Steps	If something goes wrong
F1 First open	Open link, pick language, pick nationality, choose "Just landed" or "Already in KL". Choices are saved on the phone.	Skipping nationality is allowed.
F2 Arrival	See the one recommended way to KL Sentral with the reason (price, time). Other options are shown smaller below. Follow steps, tap "I'm Here" on each.	No GPS needed. A "Learn the basics" link opens the transit pages.
F3 Plan	City Centre list sorted by distance. Open a landmark to see photo, short story, opening hours note.	Location denied: list is sorted in the suggested visiting order instead.
F4 Go	Tap "Take me there". App picks the route card from the nearest known start point. Steps shown as a checklist.	No route card for that pair: show only the "Open in Google Maps" button.

Flow	Steps	If something goes wrong
F5 Check in	On the last step, tap "I'm Here". App reads GPS. Inside the radius: gold stamp.	Outside radius or poor GPS: friendly retry message. Location denied: outline stamp only.
F6 Reward	Passport shows stamps. Coloured icon on the map. "Write a review" opens the form (stars, short text, email, consent tick).	Email fails: show "We'll try again" and keep the review saved.
F7 Postcard	Email arrives with a download link to the A5 postcard.	No postcard for that landmark yet: form still saves the review and says postcards are coming.

4. Screens

The app has 12 screens and 3 bottom tabs: Guide, Map, Passport. UI references will be added later, so build plain layouts first.

Screen	Address (URL path)	What it shows	Owner
Welcome	<code>/welcome</code>	Language, nationality, "Just landed" or "Already in KL"	Syakir
Arrival	<code>/arrival</code>	Recommended way to KL Sentral, other options smaller, start button	Syakir
Guide home	<code>/</code>	Landmark list with distance and stamp status	Syakir
Landmark detail	<code>/place/:id</code>	Photo, story, hours note, "Take me there"	Syakir
Journey	<code>/journey/:routeId</code>	Step checklist with "I'm Here" buttons, Google Maps button	Syakir
Map	<code>/map</code>	Tilted map, grey or coloured landmark icons, your position	Syakir

Screen	Address (URL path)	What it shows	Owner
Learn	<code>/learn</code> and <code>/learn/:topic</code>	Touch 'n Go, line colours, reading signs	Syakir (layout), Danial (content)
Check-in	<code>/checkin/:id</code>	GPS check, result, retry	Faris
Passport	<code>/passport</code>	Grid of 7 stamps: none, outline, gold, with dates	Danial
Review	<code>/review/:id</code>	Stars, text, email, consent tick, send	Danial
Postcard sent	<code>/postcard/:id</code>	"Check your email" message and postcard preview	Danial
Settings	<code>/settings</code>	Language, reset progress, privacy note	Faris

The `:id` part means the landmark's id from `places.json`, for example `/place/petronas`.

5. Tech stack

Everything is a website that behaves like an app on a phone, plus one small hosted backend for reviews and postcard emails. Use the latest stable version of each tool at setup time; this doc does not pin version numbers.

Part	Tool	What it does, in plain words
App framework	React + Vite	React builds screens from small reusable pieces. Vite runs and bundles the project.
Language	JavaScript (not TypeScript)	Fewer things to learn. Use JSDoc comments to describe data shapes.
Styling	Tailwind CSS	Style with short class names instead of separate CSS files.
Screen navigation	React Router	Maps a URL path like <code>/passport</code> to a screen.

Part	Tool	What it does, in plain words
Map	MapLibre GL JS	Free map library. Can tilt the map for the 2.5D look and place custom icons.
Map background	A hosted map tile service	The street map images under our icons. Pick one in week 1 (see open questions).
Saving progress	Browser localStorage	Saves stamps on the phone itself. No login needed.
Backend	Supabase	Hosted database, file storage and small server functions. Holds reviews, emails, postcard files.
Email sending	Resend (or similar), called from a Supabase function	Sends the postcard email. The secret key stays on the server, never in the app.
Languages	react-i18next	Swaps text between English and Malay from JSON files.
Install to phone	vite-plugin-pwa	Lets visitors add the site to their home screen.
Hosting	Cloudflare Pages or Vercel	Puts the site online. Makes a preview link for every pull request.
Code checks	ESLint + Prettier	Catches mistakes and keeps formatting the same across three people.
Tests	Vitest	Small automatic tests for the important logic (progress save, distance check).

Fallback: if MapLibre proves too hard in week 1, switch to Leaflet. It is simpler but cannot tilt the map, so the 2.5D look would be dropped.

Not verified: free-plan limits for Supabase, Resend, the tile service and hosting. Check each pricing page before committing.

6. Repo structure and ownership

Each person owns their own folders, so two people almost never edit the same file. That is what makes the final merge easy.

```

jalankl/
  public/
    icons/           app icons for install
    landmarks/       landmark icons (grey + colour) and photos
    signage/         real photos of transit signs
  src/
    main.jsx         app start (Faris)
    App.jsx          routes + bottom tabs (Faris)
    core/            FARIS
      progress.js    save and load stamps
      location.js    GPS reading + distance check
      checkin.js     check-in rules
      api.js         talks to Supabase
      settings.js    language, nationality, prefs
      mocks/         fake versions used in week 1
      CheckinScreen.jsx
      SettingsScreen.jsx
    guide/           SYAKIR
      WelcomeScreen.jsx
      ArrivalScreen.jsx
      GuideHomeScreen.jsx
      PlaceScreen.jsx
      JourneyScreen.jsx
      MapScreen.jsx
      LearnScreen.jsx
      components/
    rewards/         DANIAL
      PassportScreen.jsx
      ReviewScreen.jsx
      PostcardSentScreen.jsx
      components/
    data/            DANIAL
      places.json
      routes.json
      arrival.json
      learn.json
      locales/en.json
      locales/ms.json

```

shared/
 Button.jsx Card.jsx Avatar.jsx StampBadge.jsx
supabase/
 migrations/
 functions/send-postcard/
docs/
 PLAN.md
 .env.example

EVERYONE (change only by pull request)

FARIS
database table setup

a copy of this doc for the AI assistant
names of secret keys, no real values

Rule	Why
Only edit files in your own folder. Need a change elsewhere? Ask the owner or open an issue.	Avoids merge conflicts.
Screens never touch localStorage, GPS or Supabase directly. They call functions from <code>core/</code> .	One place to fix bugs. Swapping fake for real needs no screen changes.
All visible text comes from <code>locales/</code> , never typed into a screen.	Adding a language later is just a new file.
All place and route facts come from <code>data/</code> , never typed into a screen.	Danial can fix a wrong exit without touching code.
Real secret keys never go into GitHub. Only <code>.env.example</code> with empty values.	Anyone can read a public repo.

7. Data formats

Four JSON files hold all content, and one saved object holds the visitor’s progress. Agree these shapes in Phase 0 and do not change them without telling the other two.

Every value written as `TBC` below must be filled from official sources and the field day. The sample values are examples, not verified facts.

places.json (one entry per check-in spot)

```
{
  "id": "abdul-samad",
  "nameKey": "places.abdul-samad.name",
  "storyKey": "places.abdul-samad.story",
  "category": "heritage",
  "coords": [3.14861, 101.69444],
  "radius_m": 90,
  "photo": "/landmarks/abdul-samad.jpg",
  "iconGrey": "/landmarks/abdul-samad-grey.png",
  "iconColour": "/landmarks/abdul-samad-colour.png",
  "nearestStation": "masjid-jamek",
  "hasPostcard": true,
  "order": 2,
  "verified": "TBC",
  "sources": ["TBC"]
}
```

Field	Meaning
<code>id</code>	Short unique name. Used in URLs, stamps and routes. Never change it after launch.
<code>nameKey</code> , <code>storyKey</code>	Where to find the text in the language files.
<code>coords</code>	Latitude, longitude. The sample comes from the original PDF; re-check it.
<code>radius_m</code>	How close (in metres) the phone must be to count as "there". Tuned on the field day.
<code>hasPostcard</code>	<code>true</code> only for the 3 postcard landmarks.
<code>order</code>	Suggested visiting order, used when location is off.
<code>verified</code> , <code>sources</code>	Date checked and where the facts came from.

routes.json (one entry per hand-written route card)

```
{
  "id": "kl-sentral__petronas",
  "from": "kl-sentral",
  "to": "petronas",
  "mode": "train",
  "why": "routes.kl-sentral__petronas.why",
  "totalMinutes": "TBC",
  "steps": [
    { "type": "walk", "textKey": "routes.kl-sentral__petronas.s1", "photo": null },
    { "type": "board", "textKey": "routes.kl-sentral__petronas.s2", "line": "TBC", "lineColour": "TBC", "towards": "TBC" },
    { "type": "ride", "textKey": "routes.kl-sentral__petronas.s3", "stops": "TBC" },
    { "type": "exit", "textKey": "routes.kl-sentral__petronas.s4", "photo": "/signage/TBC.jpg" },
    { "type": "arrive", "textKey": "routes.kl-sentral__petronas.s5", "checkinPlace": "petronas" }
  ],
  "googleMapsUrl": "TBC",
  "verified": "TBC",
  "sources": ["TBC"]
}
```

- `mode` is `train`, `walk` or `car`. It is the one option we recommend. `why` is one sentence explaining the pick, for example that walking is faster than waiting for a car.
- Step `type` is one of `walk`, `board`, `ride`, `exit`, `arrive`. The Journey screen picks an icon from it.
- Only a step with `checkinPlace` opens the GPS check-in. Every other step advances on the tap alone.

Routes to write (12): KLIA to KL Sentral; KL Sentral to each of the 6 landmarks; then the 5 hops in the suggested order from one landmark to the next.

arrival.json

The KLIA to KL Sentral options. One is marked `"recommended": true`. Each has `id`, `nameKey`, `priceNoteKey`, `minutesNoteKey`, and a `routeId` pointing into `routes.json`. Prices and times are `TBC` until checked against the operators' own sites.

learn.json

A list of topics (touch-n-go , line-colours , reading-signs). Each topic has a titleKey and a list of blocks; a block is either { "type": "text", "key": "..." } or { "type": "photo", "src": "...", "captionKey": "..." }.

locales/en.json and locales/ms.json

Same keys in both files, different text.

```
{
  "places": { "abdu1-samad": { "name": "Sultan Abdul Samad Building",
    "story": "TBC" } },
  "routes": { "kl-sentral__petronas": { "why": "TBC", "s1": "TBC" } },
  "ui": { "imHere": "I'm Here", "takeMeThere": "Take me there" }
}
```

Saved progress (localStorage key jalankl-progress-v1)

```
{
  "version": 1,
  "prefs": { "lang": "en", "nationality": "TBC", "startedFrom": "arrival" },
  "opened": ["abdu1-samad"],
  "stamps": {
    "abdu1-samad": { "kind": "gold", "at": "2026-10-03T02:42:00Z",
  "accuracy_m": 18 }
  },
  "journeys": { "kl-sentral__petronas": { "stepIndex": 2 } },
  "reviewed": ["abdu1-samad"]
}
```

- kind is outline (opened the story or checked in without GPS) or gold (GPS confirmed). Gold never goes back to outline.
- We save how accurate the GPS was, never the coordinates.
- Every field has a default value, so an older save never crashes a newer version of the app.

8. Shared functions (the contract)

These are the only doors between the three parts of the app. Faris ships fake versions on day 3 that return made-up data, so Syakir and Danial can build screens right away. In weeks 2 and 3 the real versions replace the fakes with the same names, and no screen has to change.

Function	File	Takes	Gives back
<code>getProgress()</code>	<code>core/progress.js</code>	nothing	The whole saved progress object
<code>markOpened(placeId)</code>	<code>core/progress.js</code>	place id	Nothing. Adds an outline stamp if there is no stamp yet.
<code>addStamp(placeId, kind, accuracy_m)</code>	<code>core/progress.js</code>	place id, "outline" or "gold", number	The updated stamp
<code>getStamp(placeId)</code>	<code>core/progress.js</code>	place id	<code>null</code> , or <code>{ kind, at }</code>
<code>setJourneyStep(routeId, stepIndex)</code>	<code>core/progress.js</code>	route id, number	Nothing
<code>markReviewed(placeId)</code>	<code>core/progress.js</code>	place id	Nothing
<code>resetProgress()</code>	<code>core/progress.js</code>	nothing	Nothing
<code>onProgressChange(callback)</code>	<code>core/progress.js</code>	a function	Calls it whenever progress changes, so screens refresh
<code>getPosition()</code>	<code>core/location.js</code>	nothing	<code>{ ok, lat, lng, accuracy_m }</code> or <code>{ ok: false, reason }</code>

Function	File	Takes	Gives back
<code>distanceTo(placeId, position)</code>	<code>core/location.js</code>	place id, position	Metres as a number
<code>checkIn(placeId)</code>	<code>core/checkin.js</code>	place id	<code>{ result }</code> where result is <code>gold</code> , <code>too_far</code> , <code>poor_signal</code> , <code>no_permission</code> or <code>error</code>
<code>getPrefs()</code> and <code>setPrefs(changes)</code>	<code>core/settings.js</code>	changes object	Current preferences
<code>submitReview(review)</code>	<code>core/api.js</code>	<code>{ placeId, stars, text, email, nationality, lang, consent }</code>	<code>{ ok, postcardQueued }</code> or <code>{ ok: false, reason }</code>

How the fakes behave: `getPosition()` returns a fixed spot near Sultan Abdul Samad. `checkIn()` returns `gold` after a 2 second wait. `submitReview()` returns `{ ok: true, postcardQueued: true }` after 1 second. A switch in `.env` (`VITE_USE MOCKS=true`) picks fake or real.

Data helpers Danial provides in `data/index.js`: `getPlaces()`, `getPlace(id)`, `getRoute(id)`, `findRoute(fromId, toId)`, `getArrivalOptions()`, `getLearnTopics()`. Screens use these and never import the JSON files directly.

Shared look, agreed once: `Button`, `Card`, `StampBadge` (none, outline, gold) and `Avatar` (the neutral three-circle silhouette) live in `shared/`. Syakir builds them in Phase 0; after that, changes need a pull request that all three see.

9. Backend (Supabase)

The backend does one job: take a review, save it, and email the postcard. Everything else runs on the phone.

```
sequenceDiagram
    participant App
    participant Fn as send-postcard function
    participant DB as Database
    participant Mail as Email service
    App->>Fn: review + email + consent
    Fn->>Fn: check the fields
    Fn->>DB: save review
    Fn->>DB: find postcard file
    Fn->>Mail: send email with download link
    Fn->>DB: log the send
    Fn-->>App: ok, postcardQueued
```

The app only ever talks to the function. It never reads or writes the database directly.

Table	Columns	Notes
reviews	id, place_id, stars (1 to 5), text, email, nationality, lang, consent_at, created_at	No coordinates. No names.
postcards	place_id, file_path, photographer_credit, active	One row per postcard landmark. Files sit in a private storage bucket.
postcard_sends	id, review_id, place_id, status (sent or failed), created_at	Lets us retry failed sends and count postcards.

Rules inside the function

- Reject if consent is not ticked, the email looks invalid, stars are not 1 to 5, or the text is over 500 characters.
- One postcard per email per landmark. A repeat request re-sends the same link instead of making a new row.
- Limit how many requests one phone can make per hour, so nobody can spam the email service.
- The download link is time-limited (a signed link), so the postcard file is not open to the whole internet.
- If the landmark has no postcard yet, still save the review and answer `postcardQueued: false`.

Security setup

- Turn on Row Level Security for all three tables with no public access. This Supabase setting blocks anyone from reading the tables using the key that ships inside the app.
- The email service key and the Supabase service key live only in the function's secret settings.
- The honest limit: GPS on a website can be faked by a determined person. For the MVP, stamps are a souvenir, not a prize, so we accept this. Stronger checks can come later.

Privacy (Malaysia's Personal Data Protection Act, PDPA)

- Email and nationality are personal data. The review form needs an unticked consent box with a plain sentence saying what we collect, why, and how to ask for deletion.
- Add a short privacy page linked from the form and from Settings.
- Pick a contact email for deletion requests before launch.
- This is not legal advice. Have someone qualified check the wording before real tourists use it.

10. Feature rules

These are the exact behaviours to build, so nobody has to guess.

Check-in

```
stateDiagram-v2
    [*] --> Asking: tap I'm Here on last step
    Asking --> NoPermission: location denied
    Asking --> Reading: location allowed
    Reading --> PoorSignal: accuracy worse than 50 m
    Reading --> TooFar: outside radius
    Reading --> Gold: inside radius
    PoorSignal --> Reading: retry
    TooFar --> Reading: retry
    NoPermission --> Outline: continue without GPS
    Gold --> [*]
    Outline --> [*]
```

Rule	Value	Note
Good enough GPS accuracy	50 m or better	From the original plan. Tune on field day.
Default radius	90 m	Per landmark in <code>places.json</code> . Tall towers near KLCC and Merdeka 118 may need more.
Reading time	Take up to 3 readings over about 10 seconds, use the most accurate	Replaces the original plan's 90 second wait, because the visitor already tapped to confirm.
Explainer before asking	Always show one sentence on why we need location before the phone's own pop-up	People say yes more often when they know why.
Location use	Only while the Check-in, Guide home or Map screen is open. Stop when the app is hidden.	Saves battery.
Impossible jumps	Ignore a gold check-in less than 2 minutes after another gold at a different landmark	The only anti-cheat in the MVP.

Stamps and colour

- Three states per landmark: none, outline, gold.
- Outline: opened the landmark story, or finished the journey without GPS.
- Gold: GPS confirmed. Gold is permanent and shows the date.
- The map icon and the passport badge both read from `getStamp()`. Grey icon for none or outline, colour icon for gold.

Postcards

- The review button appears only after a gold stamp for that landmark.
- Postcard is an A5 design made in InDesign, exported as a print-quality PDF plus a smaller JPG preview.
- Photos are real photographs by a hired photographer. No AI-generated images.
- The photographer credit appears on the postcard and in the email.

Best-route advice

- Each route card shows one recommended option and one sentence on why. No list of alternatives, except on the Arrival screen where other options are shown smaller.
- If the walk between two landmarks is about 12 minutes or less, the route card is a walking route.
- Every route card has an "Open in Google Maps" button for live times.
- All times shown are labelled as estimates.

Languages

- English and Malay at launch. The language picker reads the list of files in `locales/`, so adding Mandarin later needs no code change.
- Arabic reads right to left and will need layout changes. Plan that as its own task after the MVP.

Avatar

- A neutral silhouette made of three simple circles. Drawn in code (SVG), one colour, no face, no gender.

11. Task lists

Each task becomes one GitHub issue and one branch. "Done when" is the test the reviewer uses before approving the pull request. P0 is Phase 0; W1 to W4 are the weeks.

Faris: core and backend

ID	Task	When	Done when
F1	Create repo, Vite + React + Tailwind + Router, ESLint, Prettier, folder layout, README	P0	Anyone can clone, run <code>npm install</code> and <code>npm run dev</code> , and see 3 empty tabs
F2	Hosting with preview link per pull request; protect <code>main</code>	P0	A test pull request shows a working preview link
F3	Fake versions of every function in section 8, with the <code>VITE_USE MOCKS</code> switch	P0	Syakir and Danial can call every function and get sample data

ID	Task	When	Done when
F4	Real <code>progress.js</code> with defaults, change listener and tests	W1	Stamps survive a reload; an empty or broken save does not crash
F5	<code>settings.js</code> and Settings screen (language, reset, privacy link)	W1	Changing language updates every screen
F6	Real <code>location.js</code> : permission explainer, reading, distance maths, stop when hidden	W2	Shows correct distance to a known point when tested outdoors
F7	Real <code>checkin.js</code> and Check-in screen with all states in section 10	W2	Each state can be triggered with a debug menu and looks right
F8	Supabase project, 3 tables, security rules, storage bucket	W2	Tables exist; reading them with the public key is blocked
F9	<code>send-postcard</code> function with all rules in section 9	W3	A test review sends a real email with a working link
F10	Real <code>api.js</code> wired to the function, with error handling	W3	Review screen works with mocks switched off
F11	Install-to-phone setup (app name, icons, offline shell)	W3	"Add to home screen" works on Android and iPhone
F12	Privacy page and consent wording	W3	Linked from Review and Settings
F13	Final wiring, bug fixing, release	W4	Full "Done means" path passes on two phones

Syakir: guide and map

ID	Task	When	Done when
S1	Shared components: Button, Card, StampBadge, Avatar, bottom tabs	P0	A sample page shows all of them in every state
S2	Welcome screen: language, nationality, start choice	W1	Choices are saved through <code>setPrefs()</code> and skipped on next open
S3	Guide home: list from <code>getPlaces()</code> , distance, stamp status, sorting	W1	Sorted by distance with location, by <code>order</code> without

ID	Task	When	Done when
S4	Landmark detail: photo, story, hours note, "Take me there"; calls <code>markOpened()</code>	W1	Opening a landmark gives an outline stamp
S5	Journey screen: step checklist, icons per step type, "I'm Here", progress saved per step, Google Maps button	W2	Closing and reopening returns to the same step
S6	Arrival screen: recommended option large, others small, starts the journey	W2	KLIA to KL Sentral can be followed end to end
S7	Map screen: tilted map, landmark icons, your position, tap icon to open detail	W2	All 7 icons sit in the right places
S8	Colour unlock: icon switches to colour on gold, updates live	W3	Earning a gold stamp changes the icon without a reload
S9	Learn screens from <code>learn.json</code>	W3	3 topics readable in both languages
S10	Phone polish: small screens, big tap targets, loading and empty states	W3	Usable one-handed on a small Android phone
S11	Field test fixes for guide and map	W4	Top issues from the test are closed

Danial: rewards and content

ID	Task	When	Done when
D1	Draft <code>places.json</code> for 7 spots and the data helper file <code>data/index.js</code>	P0	<code>getPlaces()</code> returns 7 entries with sample values
D2	<code>en.json</code> and <code>ms.json</code> skeleton with all UI keys	P0	No screen shows a raw key name
D3	Passport screen: 7 badges with dates, count, link to map	W1	Shows none, outline and gold correctly from <code>getProgress()</code>
D4	Landmark stories and hours notes, both languages, with sources	W1	Each story has a source and a checked date

ID	Task	When	Done when
D5	<code>routes.json</code> : 12 route cards from official operator maps	W2	Every route has sources; nothing is from memory
D6	<code>arrival.json</code> and <code>learn.json</code> content	W2	Prices and times come from operator sites, with the date checked
D7	Field day: walk every route, note exits, walking times, take signage photos, record GPS accuracy at each landmark	W2	Every <code>TBC</code> in places and routes is filled
D8	Review screen: stars, text, email, consent tick, validation, calls <code>submitReview()</code>	W3	Cannot send without consent; shows clear errors
D9	Postcard sent screen with preview	W3	Shows the right postcard or the "coming soon" message
D10	Photographer brief, photo shoot for 3 landmarks, written permission to use the photos	W1 to W3	3 photos delivered with usage rights in writing
D11	InDesign A5 template; export 3 postcards as PDF and JPG; hand files to Faris	W3	Files uploaded to the storage bucket
D12	Landmark icons, grey and colour, 7 each	W2	14 image files in <code>public/landmarks/</code>
D13	Test scripts, recruit 3 to 5 testers, run the field test, write a one-page report	W4	Report lists top issues by how often they happened

Load check: Faris carries the hardest code, Syakir the most screens, Danial the least code but all the real-world legwork. If Danial's coding tasks (D3, D8, D9) fall behind, Syakir takes D3 and Faris takes D8.

12. Phases and weekly demos

Five phases, each ending with a demo on a real phone using the preview link. If a phase slips, cut from the "cut first" column; never push the demo goal to a later week.

Phase	Goal	Demo at the end	Cut first if late
Phase 0 (days 1 to 3)	Agree sections 7 and 8 together. Repo, hosting, fakes and shared components ready.	Empty app with 3 tabs opens on everyone's phone from a link	Nothing. Do not start features before this is done.
Week 1: Screens	Every screen exists and runs on fake data	Click through the whole app: welcome, list, detail, passport, settings	Settings screen polish
Week 2: Go	Real routes, real GPS, map, backend tables, field day	KLIA to KL Sentral journey works; check-in gives gold at one real landmark	Muzium Negara; map tilt
Week 3: Reward	Colour unlock, review, postcard email, learn pages, install to phone	Full loop on a phone, and the stamp survives closing the app	Learn pages down to 1 topic; postcards down to 1 landmark
Week 4: Prove and ship	Test with 3 to 5 tourists, fix top issues, go public	Live link plus a one-page test report	New features. Only fixes this week.

Weekly rhythm

- Monday: 15 minute call. Each person says what they will finish this week and what blocks them.
- Every 2 days: merge what is finished, even if small.
- Friday: demo on real phones, then move unfinished tasks and decide cuts together.

The integration moments (when separate work gets wired together)

1. End of Phase 0: everyone's code calls the fakes.
2. Middle of week 2: real progress and location replace the fakes. Syakir and Danial change nothing.
3. Middle of week 3: real backend replaces the fake `submitReview()`. Danial changes nothing.
4. Start of week 4: mocks switched off for good, full path tested.

13. GitHub workflow

One rule above all: nobody pushes straight to `main`. Everything goes through a small pull request that one teammate checks.

flowchart LR

```

A[Pick an issue] --> B[New branch<br/>from main]
B --> C[Code + commit]
C --> D[Open pull request]
D --> E[Checks pass +<br/>preview link]
E --> F[Teammate reviews]
F --> G[Merge to main]
G --> H[Delete branch]

```

Topic	Rule
Branch names	<code>area/short-task</code> , for example <code>guide/journey-screen</code> , <code>core/checkin</code> , <code>rewards/passport</code> , <code>data/routes</code>
Commit messages	Start with the task ID: <code>S5 add step checklist</code>
Pull request size	Small. Aim for one task, under roughly 300 changed lines.
Pull request text	What changed, how to test it, a screenshot if it is a screen
Review	One approval needed. The reviewer opens the preview link on a phone and tries it.
Before starting work each day	<code>git pull</code> on <code>main</code> , then update your branch from <code>main</code>
Merge conflicts	The branch owner fixes them. If it touches someone else's folder, fix it together on a call.
Board	GitHub Projects with 4 columns: To do, Doing, Review, Done
Labels	<code>faris</code> , <code>syakir</code> , <code>danial</code> , <code>bug</code> , <code>content</code> , <code>blocked</code>
Secrets	Never commit <code>.env</code> . It is listed in <code>.gitignore</code> from F1.

Automatic checks on every pull request (GitHub Actions, set up in F2): install, run ESLint, run tests, build. A red cross means fix it before asking for review.

14. Working with an AI coding assistant

Give the assistant one task at a time, with the rules of this doc pasted in, and test what it writes on a phone before merging. This section is written for any assistant; nothing here depends on a specific model.

Setup: export this doc as Markdown and save it in the repo as `docs/PLAN.md`. Point the assistant at that file at the start of every session, so all three of you work from the same rules.

Prompt template

You are helping build JalanKL, a phone web app. Read `docs/PLAN.md` first.

My task: [task ID and name from section 11]

Done when: [copy the "Done when" cell]

Rules you must follow:

- Stack: React + Vite, plain JavaScript, Tailwind CSS, React Router.
- Only create or edit files inside: [my folder, e.g. `src/guide/`]
- Do not touch `localStorage`, GPS or Supabase directly. Use the functions in section 8 exactly as named. If one is missing, stop and tell me.
- All visible text comes from `src/data/locales` via `react-i18next`.
- All place and route facts come from the helpers in `src/data/index.js`.
- Do not add new libraries without asking me first.
- Do not invent transit facts, prices, coordinates or station names. Use the value "TBC" if a fact is missing.
- Build for phone screens first. Tap targets at least 44 px tall.

Before writing code, list the files you will create or change and wait for my OK. After writing, explain in simple words what each file does and how I can test it.

Habits that prevent most problems

Do	Why
One task per chat session	Long sessions drift away from the rules

Do	Why
Ask it to explain anything you do not understand before merging	You will have to fix this code later without it
Read every changed file in the pull request	Assistants sometimes edit files you did not ask about
Reject new libraries unless all three agree	Each one adds size and things that can break
Never paste real secret keys into a chat	Use the names from <code>.env.example</code> instead
Never accept a transit fact, price or coordinate from the assistant	It can sound sure and still be wrong. Facts come from operators and the field day only.
Ask for tests on logic tasks (F4, F6, F7, F9)	Proof it works, and it stays working later
Run it on a real phone, not only the laptop browser	GPS, install and small screens only show problems on a phone

15. Testing and field day

Test on two real phones every Friday: one mid-range Android and one iPhone. GPS only works on a secure (https) address, so always test from the preview link, not from a laptop address.

Field day (week 2, all three if possible)

- ☐ Ride KLIA to KL Sentral following only our steps; fix any step that confused you
- ☐ Walk or ride every one of the 12 routes; write down the real exit name and walking minutes
- ☐ Photograph the key signs at each station for the route steps (check whether photography is allowed there)
- ☐ At each of the 7 spots, stand where a tourist would and note the GPS accuracy the app shows, in 3 positions
- ☐ Set each landmark's `radius_m` from those readings
- ☐ Fill `verified` with the date and `sources` for every place and route

Friday phone checklist

- ☐ Opens from the link in under about 3 seconds on mobile data
- ☐ Welcome choices are remembered after closing the app

- ☐ List sorts by distance with location on, and by suggested order with location off
- ☐ Journey returns to the same step after closing the app
- ☐ Check-in shows the right message for: allowed and near, allowed and far, poor signal, denied
- ☐ Gold stamp appears in the passport and the map icon turns to colour
- ☐ Review cannot be sent without the consent tick
- ☐ Postcard email arrives and the link opens
- ☐ Every screen reads correctly in English and Malay
- ☐ Nothing important is hidden behind the phone's bottom bar or notch

Field test with tourists (week 4)

- 3 to 5 people who do not know KL well. Give them the link and one goal: get from Merdeka Square to Petronas Twin Towers and collect the stamp, using only the app.
- Watch without helping. Write down where they stop, tap the wrong thing, or open another app.
- Pass mark, kept from the original plan: 4 of 5 finish without help.

Numbers to watch after launch

Question	Measure	Starting target
Is it useful for getting around?	Visitors who open at least one journey	40% or more
Does it get people to real places?	Visitors with at least one gold stamp	15% or more
Do rewards work?	Gold stampers who send a review	No target yet. Set one after 2 weeks of data.

The first two targets come from the original plan. Counting them needs simple, privacy-safe event tracking, which is a stretch task for week 4 if time allows.

16. Risks and open questions

The two biggest risks are trying to build too much, and sending a tourist the wrong way with bad transit facts.

Risk	Level	What we do about it
Scope is still large for 3 junior developers in 4 weeks	High	The "cut first" column in section 12. Friday decisions are final.
Wrong exit, line or direction in a route card	High	Official sources plus field day, a checked date on every route, Google Maps button on every card
AI-written code nobody on the team understands	High	The habits in section 14; reviewer must be able to explain the change
GPS is unreliable near tall towers and indoors	Medium	Radius set per landmark, accuracy filter, retry, outline stamp as a fallback
Browser clears saved progress (Safari may do this for sites not used for a while)	Medium	Ask visitors to add the app to their home screen after the first stamp
Postcard emails land in spam	Medium	Set up the sending domain properly in the email service; test with Gmail and Outlook
Photographer delays or unclear photo rights	Medium	Start D10 in week 1; get usage permission in writing
Personal data handling under PDPA	Medium	Consent tick, privacy page, deletion contact, qualified review before launch
Free plan limits on hosted services	Low	Check pricing pages in Phase 0

Open questions for the team

- ☐ Are the assumptions in section 2 right (languages, 3 postcard landmarks, 4 weeks, Danial's coding load)?
- ☐ Which map tile service do we use? Compare 2 or 3 in Phase 0 on price and on how good KL looks.
- ☐ What email address sends the postcards, and do we own a domain for it?
- ☐ Who is the contact for data deletion requests?
- ☐ Is photography allowed inside the stations we need signage photos from?
- ☐ What is the postcard photo budget in total, and who pays?

- ☐ When KrackedDev joins, what do they need from us: the repo, a demo, this doc?
- ☐ UI and UX references: when they arrive, Syakir updates the shared components first so every screen follows.

After the MVP, in rough order: more languages (Mandarin, Arabic, an Indian language), more neighbourhoods, real 3D models for hero landmarks on the map, an automatic route finder, accounts so progress moves between phones, partner rewards.